

M04 Shell Programming (1), Linux

DASAR TEORI

4.1 Shell Script

Shell script dibuat dengan teks editor. Umumnya menggunakan ekstensi “.sh”. Script selalu diawali dengan komentar, yang dimulai dengan tanda #, disambung dengan ! dan nama shell yang digunakan.

Contoh:

```
#!/bin/sh

# Program shell

#

var1=x

var2=8
```

#!/bin/sh → Awal dari program shell, komentar awal ini akan dibaca oleh system, kemudian system mengaktifkan program shell (/bin/sh) yang tertera di situ. Program shell dapat dipilih, misalnya /bin/csh, /bin/ksh dan lainnya

.# → komentar, sebagai dokumentasi, baris ini akan diabaikan oleh program shell

var1=x dan var2=8 → Penggunaan variable (assignment), tidak boleh ada spasi di antara nama variable dan konstanta.

4.2 Variabel

Variable shell adalah variable yang dapat mempunyai nilai berupa nilaiString. Tata penulisan variable adalah sebagai berikut :

```
nama_var=nilai_var
```

Variable dimulai dengan alfabet, dan dapat disusul dengan alfanumerik dan karakter lain. Variabel dapat ditulis dalam huruf kecil atau huruf besar atau campuran keduanya. Shell membedakan huruf besar dan huruf kecil (case sensitive), contoh :

```
MYVAR=polman
n=5
```

Pemberian nilai variable tidak boleh dipisahkan dengan spasi. contoh :

```
MYVAR =poltek ##error  
MYVAR = poltek ##error
```

Untuk melihat nilai/isi dari sebuah variable, gunakan tanda \$ di depan nama variable tersebut. Pada shell, instruksi echo dapat menampilkan isi variable tersebut, contoh :

```
MYVAR =poltek  
echo $ MYVAR  
Gaji=4500000  
echo $Gaji  
echo $MYVAR $Gaji
```

Bila menggunakan string yang terdiri dari lebih dari satu kata, maka string tersebut harus berada dalam tanda kutip atau apostrof, contoh :

```
VPT=poltek  
VPT2="poltek manufaktur ASTRA"
```

4.3 Membaca Keyboard

Nilai variable dapat diisi melalui keyboard (stdin) dengan perintah `read`.

4.4 Parameter

Sebuah program shell dapat mempunyai parameter sebanyak 9 buah dan direpresentasikan melalui variable khusus yaitu variable \$!, \$2, \$3, \$4, \$5, \$6, \$7, \$8 dan \$9. Nama program shell (nama script) direpresentasikan melalui variable \$0.

Jumlah parameter dinyatakan sebagai \$#. Bila tidak memberikan parameter, maka nilai \$# adalah 0.

Shell variable \$* menyatakan seluruh string yang menjadi parameter / argumen sebuah script (\$@ mempunyai arti yang sama). \$\$ menyatakan nomor proses id (pid) dari script yang dijalankan. Pid ini akan terus berubah (umumnya) menaik, setiap kali proses berjalan.

4.5 Status Exit

Setiap program setelah selesai dieksekusi akan memberikan informasi melalui variable spesial \$? . Indikasi yang diberikan adalah :

- Bila program berakhir dengan sukses, \$? = 0
- Bila program berakhir dengan error, \$? ≠ 0

Nilai dari status exit dapat dilihat melalui instruksi echo \$?

4.6 Konstruksi If

Format Umum :

```

if perbandingan
then
    instruksi 1
    instruksi 2
    ...
fi

```

4.7Konstruksi If then else

Format Umum :

```

if perbandingan
then
    instruksi 1
    instruksi 2
    ...
else
    instruksi 1
    instruksi 2
    ...
fi

```

4.8Konstruksi if then else if

```

if instruksi1
then
    instruksi1.1
    instruksi1.2
    .....
elif instruksi2
then
    instruksi2.1
    instruksi2.2
    .....
else
    instruksi3.1
    instruksi3.2
    .....
fi

```

Bila status exit tidak sama dengan 0, maka kondisi menjadi FALSE dan instruksi setelah else akan dijalankan.

4.9 Instruksi TEST

Instruksi test digunakan untuk memeriksa kondisi dari sebuah ekspresi. Ekspresi terdiri dari factor dan operator yang dipisahkan oleh spasi. Hasil test akan memberikan nilai berupa status exit, yaitu 0 bila ekspresi sesuai, bila tidak maka hasil adalah $\neq 0$.

- Operator untuk test

Operator	0 atau TRUE, jika
string1 = string2	Identical
string1 != string2	Not identical
-n string	String is not null
-z string	String is null

- Test untuk files dan directory

Operator	0 atau TRUE, jika
-f namafile	File ada, file biasa
-d namafile	File ada, file adalah direktori
-r namafile	File dapat dibaca
-w namafile	File dapat ditulis
-x namafile	File adalah executable
-s namafile	File ada dan tidak kosong
-w namafile	File dapat ditulis

Untuk memudahkan pembacaan (readability), test dapat ditulis dengan [ekspresi].

[sebenarnya adalah nama lain dari test, bedanya [akan mencari kurung penutup] pada akhir ekspresi yang harus dipisahkan oleh spasi.

4.10 Logical && Dan || (SHELL LEVEL)

Notasi && dan || digunakan untuk menggabungkan instruksi shell sebagai alternatif untuk if then else. Notasi && dan || sering ditemukan adalah shell script system administrator untuk menjalankan routine dari system operasi.

- instruksi1 && instruksi2
shell akan mengeksekusi instruksi1, dan bila exit status instruksi1 adalah FALSE, maka hasil dari AND tersebut sudah pasti sama dengan FALSE, sehingga instruksi2 tidak mempunyai pengaruh lagi. Oleh karena itu, instruksi2 tidak dijalankan. Sebaliknya bila hasilinstruksi1 adalah TRUE(0), maka instruksi2 dijalankan
- instruksi1 || instruksi2
shell akan mengeksekusi instruksi1, bila exit status adalah TRUE(0), hasil dari operasi OR tersebut sudah pasti menghasilkan TRUE, terlepas dari hasil eksekusi instruksi2. Oleh karena itu instruksi2 tidak perlu dijalankan. Bila hasil instruksi1 adalah FALSE, maka instruksi2 akan dijalankan.

4.11 OPERATOR BILANGAN BULAT UNTUK TEST

Untuk membandingkan 2 buah bilangan, test memerlukan operator yang berbeda dengan string.

Operator	0 atau TRUE, jika
i1 -eq i2	Bilangan sama
i1 -ge i2	Lebih besar atau sama dengan
i1 -gt i2	Lebih besar
i1 -le i2	Lebih kecil atau sama dengan
i1 -lt i2	Lebih kecil
i1 -ne i2	Bilangan tidak sama

4.12 OPERATOR LOGICAL (TEST LEVEL)

Logical operator terdiri dari AND, OR dan NOT. Operator ini menggabungkan hasil ekspresi sebagai berikut :

NOT : symbol !

	!
True	False
False	True

AND : symbol -a

V1	V2	V1 -a V2
False	False	False
False	True	False
True	False	False
True	True	True

OR : symbol -o

V1	V2	V1 -o V2
False	False	False
False	True	True
True	False	True
True	True	True

4.13 HITUNGAN ARITMETIKA

Tipe dari variable SHELL hanya satu yaitu STRING. Tidak ada tipe lain seperti Numerik, Floating, Boolean atau lainnya. Akibatnya variable ini tidak dapat membuat perhitungan aritmetika, misalnya :

```
A=5
```

```
B=$A +1 ## error
```

UNIX menyediakan utilitas yang bernama **expr** yaitu suatu utilitas yang melakukan aritmetika sederhana.

4.14 INSTRUKSI EXIT

Program dapat dihentikan (terminated/selesai) dengan instruksi exit. Sebagai nilai default program tersebut akan memberikan status exit 0.